
Lab Review: Successful Path

One successful path:

- ▶ site runs without console errors
- ▶ interaction works
- ▶ bug report explains the issue
- ▶ fix is connected to evidence
- ▶ verification proves the result

Now we will ask:

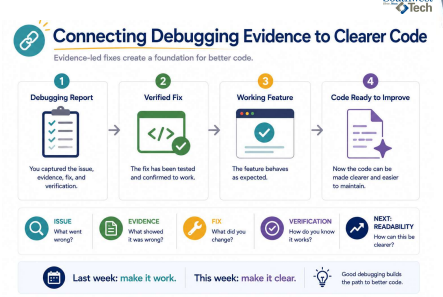
Can another person understand the code?



4

Connecting Debugging Evidence to Clearer Code

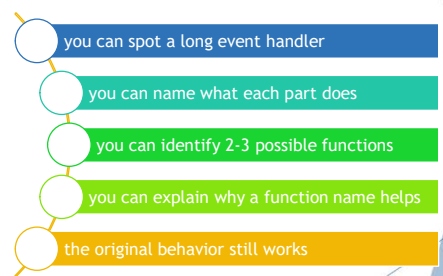
Evidence-led fixes create a foundation for better code.



Week 6 Success Path

5

What Counts as Success Today



- you can spot a long event handler
- you can name what each part does
- you can identify 2-3 possible functions
- you can explain why a function name helps
- the original behavior still works

6



Debugging vs. Refactoring

Two different questions. One important goal.

DEBUGGING



Why did this fail?

Find the cause.
Fix the problem.

REFACTORING



How can this be clearer while still working?

Improve structure.
Keep behavior the same.

PRESERVE BEHAVIOR

 **Change structure.**

 **Preserve behavior.**

Debugging To Refactoring

7



What We Will Use Today

```

mirror_mod = modifier_ob
mirror_mod.mirror_object = mirror_mod.mirror_object

operation = "MIRROR_X"
mirror_mod.use_x = True
mirror_mod.use_y = False
mirror_mod.use_z = False
operation = "MIRROR_Y"
mirror_mod.use_x = False
mirror_mod.use_y = True
mirror_mod.use_z = False
operation = "MIRROR_Z"
mirror_mod.use_x = False
mirror_mod.use_y = False
mirror_mod.use_z = True

#selection at the end - add
obj.select-1
obj.select-1
context.scene.objects.active
("Selected" + str(modifier_mirror_object.select-0
bpy.context.selected_object.name) + ".select-1")
int("please select exactly one X mirror")

-- OPERATOR CLASSES --

types.Operator):
X mirror to the selected object.mirror_mirror_X
error X"

context.active_object is no

```

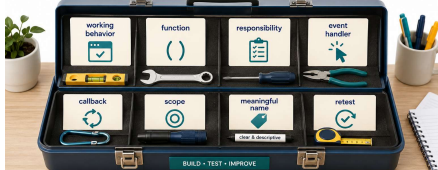
- ▶ working behavior
- ▶ function
- ▶ responsibility
- ▶ event handler
- ▶ callback
- ▶ scope
- ▶ meaningful name
- ▶ retest

8



Today's Structured JavaScript Toolbox

Tools for building clear, organized, and reliable behavior.



Use these tools together. Write code that works, is easy to understand, and is easy to maintain.

 **Clear purpose**

 **Focused responsibilities**

 **Responds to actions**

 **Test and confirm**

 **Refine and improve**

Structured JavaScript Toolbox

9



What We Will “Skip” For Now

- ▶ modules
- ▶ classes
- ▶ complex objects
- ▶ advanced scope rules
- ▶ build tools
- ▶ framework patterns



10



Advanced JavaScript Structure Topics Parked for Later

Important tools. Not today. We will return to them.

PARKED FOR LATER

modules

classes

complex objects

advanced scope

build tools

framework patterns

Today's Focus:

Clear named functions in one JavaScript file.

Why we focus:

Build strong habits first. Add advanced tools later.

Master the basics now.

Build confidence and clarity.

Advanced topics wait in easily later.

Parked For Later

11

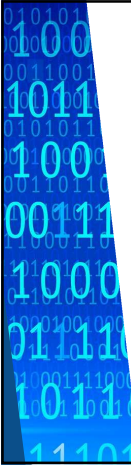


Code Can Work And Still Be Hard To Read

Working but messy code often has:

- ▶ one long event handler
- ▶ repeated output text
- ▶ several decisions in one place
- ▶ vague names
- ▶ hidden responsibilities

*The browser may be happy.
The next human reader may not be.*



12

Code That Works but Is Hard to Read

It works, but it is doing too many things at once.

One Long Event Handler

```

button.addEventListener('click', function () {
  // get name = localStorage.getItem('name');
  if (name === null) {
    message.textContent = "Please enter your name.";
    message.classList.add = "warning";
  } else {
    // get plan = localStorage.getItem('plan');
    if (plan === null) {
      // if plan === "basic" {
      //   total = 10;
      // } else if (plan === "standard") {
      //   total = 20;
      // } else if (plan === "premium") {
      //   total = 30;
      // }
      total = total + (Number(localStorage.getItem('minutes')) || 0);
      localStorage.setItem('name', name);
      localStorage.setItem('plan', plan);
      localStorage.setItem('minutes', total.toString());
      message.textContent = "Here is your plan.";
      message.classList.add = "green";
      button.disabled = true;
    }
  }
});

```

Browser Output (Working Correctly)

Plan Calculator

Your name:

Plan:

Choose a plan:

Standard Plan

Extra minutes:

Total: **\$35.00**

Here is your plan.

Working But Hard To Read

13

Function As A Named Responsibility

What job does this code do?

What information does it need?

What result does it produce?

Is the name honest?

Good names make code easier to read aloud.

14

Turning a Responsibility into a Named Function

Name the job. Clarify the purpose.

RESPONSIBILITY

choose the study plan

This part of the code decides which plan to use.

FUNCTION

chooseStudyPlan(minutes)

f()

This function takes the minutes and returns the plan to use.

What job? Choose the study plan based on the minutes.

What input? The number of minutes to study.

What result? The plan name (basic, standard, premium).

Good names make code easier to read.

Function As Named Responsibility

15

Callback: A Function Used Later

In browser code, a callback is often:

- ▶ a function
- ▶ handed to another piece of code
- ▶ used later when something happens

Example:

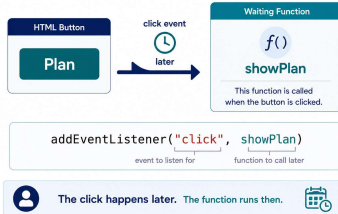
```
'button.addEventListener("click",  
showPlan);
```

The click happens later.
The function runs then.

16

Callback: A Function Used Later

We register the function now. It runs when the event happens.



Callback Used Later

17

Scope: Where A Name Can Be Seen

Scope asks:

Where can this name be used?

Beginner rule:

- ▶ keep shared page elements near the top
- ▶ keep temporary values inside functions
- ▶ avoid depending on names from everywhere

Clear scope makes debugging easier.

18



Scope: Where Names Can Be Used

Names exist in the places where they are defined.

Shared Page Elements

- minutesInput**
Input field where the user enters minutes.
- planButton**
Button the user clicks to plan.
- message**
Area where messages are shown.

Inside the Function

```

f() showPlan()
      minutes
      plan
          
```

- minutes**
Local name that holds the number of minutes.
- plan**
Local name that holds the plan to use.

Scope asks: where can this name be used?

Scope Basics

19



AI Can Explain Structure, Not Replace Your Refactor

Useful:

- explain what a function does
- explain why a name is clearer
- suggest possible responsibilities to look for

Not useful:

- "rewrite my whole assignment"
- "make this perfect"
- pasted code you cannot explain

20



AI as an Explainer for Code Structure

You write the code. AI helps you understand and improve the structure.

Student-Owned Code

Your refactored function (example):

```

function chooseStudyPlan(minutes) {
  // decide which plan fits the time
  let plan;
  if (minutes < 20) {
    plan = "basic";
  } else if (minutes < 40) {
    plan = "standard";
  } else {
    plan = "premium";
  }
  return plan;
}
          
```

You own the code. You make the decisions.

AI Explanation (for Understanding)

What this function does
It looks at the number of minutes and returns a plan name: basic, standard, or premium.

Why this name is clearer
chooseStudyPlan() states the purpose. Anyone reading the code knows it selects a plan based on the minutes.

Possible responsibilities to look for

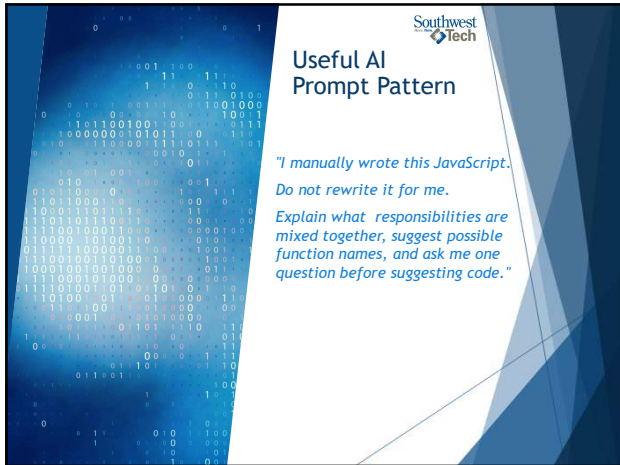
- Input validation (e.g., negative minutes)
- Plan rules (thresholds or plan names)
- Returning the plan (not showing it)
- Keeping UI updates outside this function

Boundary: Explain structure. Do not replace the refactor.

You refactor. AI explains.

AI as Structure Explainer

21

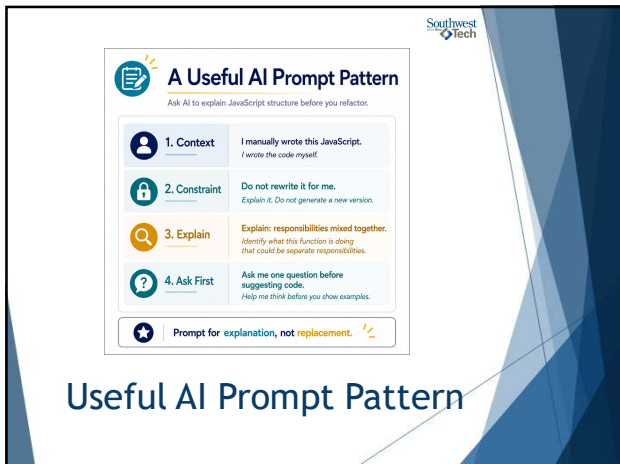


Southwest
Tech

Useful AI Prompt Pattern

*"I manually wrote this JavaScript.
Do not rewrite it for me.
Explain what responsibilities are
mixed together, suggest possible
function names, and ask me one
question before suggesting code."*

22



Southwest
Tech

A Useful AI Prompt Pattern

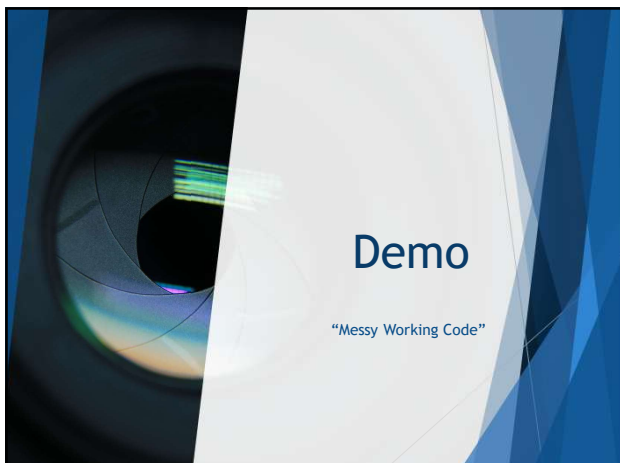
Ask AI to explain JavaScript structure before you refactor.

1. Context	I manually wrote this JavaScript. I wrote the code myself.
2. Constraint	Do not rewrite it for me. Explain it. Do not generate a new version.
3. Explain	Explain: responsibilities mixed together. Identify what this function is doing that could be separate responsibilities.
4. Ask First	Ask me one question before suggesting code. Help me think before you show examples.

★ Prompt for explanation, not replacement. 🗨️

Useful AI Prompt Pattern

23



Demo

"Messy Working Code"

24



What to Watch For

- ▶ the feature works
- ▶ the event handler is long
- ▶ decisions are mixed with output
- ▶ the code is harder to read aloud

*Working is the starting point.
Clarity is the improvement.*

25



Demo: Messy Working Code

Study Planner

How many minutes do you have?
45 min

Plan My Study

Your Plan

- Great choice! You have 45 minutes. Here is your plan:
- Review key ideas (15 min)
- Practice problems (20 min)
- Quick summary (10 min)

You're all set. Stay consistent!

One Long Event Handler

- feature works
The page responds and shows a plan.
- handler is long
A lot of code is doing many things.
- decisions mixed with output
Logic, messages, and formatting are tangled.
- ready for refactor
Responsibilities can be separated and named.

26



Find The Responsibilities

Possible responsibilities:

- ▶ read the minutes input
- ▶ decide which plan fits
- ▶ update the message on the page
- ▶ respond to the button click

Possible function names:

- ▶ `getMinutesAvailable()`
- ▶ `chooseStudyPlan(minutes)`
- ▶ `showPlan()`

27

Southwest Tech

Hidden Responsibilities → Function Names

Break a long handler into clear, named jobs.

Hidden in One Long Handler	Clear Function Names
read minutes input Get the number of minutes from the page.	getMinutesAvailable() Get the number of minutes from the input.
choose study plan Decide which plan fits the minutes available.	chooseStudyPlan(minutes) Return the plan that fits the minutes available.
update message Show the plan and next steps on the page.	showPlan() Update the message with the plan and next steps.
respond to click Run everything when the button is clicked.	

★ Name the job before moving the code.

Responsibilities To Functions

28

Southwest Tech

Lab

First refactor pass

Your goal:

- ▶ choose existing working JavaScript
- ▶ identify repeated or mixed logic
- ▶ create at least 2-3 functions
- ▶ give each function a clear name
- ▶ retest the behavior

29

Southwest Tech

Evidence

Preserve evidence:

- ▶ original behavior still works
- ▶ JavaScript has named functions
- ▶ function names match their jobs
- ▶ repeated logic is reduced where possible
- ▶ you can explain what changed

30



Closing Success Check

Today:

- ▶ working code became a starting point
- ▶ hidden responsibilities became visible
- ▶ functions gave responsibilities names
- ▶ behavior still had to be verified

Next:

We extract the functions and compare messy vs clean code.

31



Thank you!

Have Fun!

32
